

RESEARCH ARTICLE

A distributed tracing pipeline for improving locality awareness of microservices applications

Carmine Colarusso | Assunta De Caro | Ida Falco | Lorenzo Goglia | Eugenio Zimeo 

Department of Engineering, University of Sannio, Benevento, Italy

Correspondence

Eugenio Zimeo, Department of Engineering, University of Sannio, Via Traiano 3, Benevento, 82100, Italy.
Email: zimeo@unisannio.it

Funding information

Ministero dell'Università e della Ricerca, Grant/Award Number: 201752ENYB_002

Abstract

The microservices architectural style aims at improving software maintenance and scalability by decomposing applications into independently deployable components. A common criticism about this style is the risk of increasing response times due to communication, especially with very granular entities. Locality-aware placement of microservices onto the underlying hardware can contribute to keeping response times low. However, the complex graphs of invocations originating from users' calls largely depend on the specific workload (e.g., the length of an invocation chain could depend on the input parameters). Therefore, many existing approaches are not suitable for modern infrastructures where application components can be dynamically redeployed to take into account user expectations. This paper contributes to overcoming the limitations of static or off-line techniques by presenting a big data pipeline to dynamically collect tracing data from running applications that are used to identify a given number k of microservices groups whose deployment allows keeping low the response times of the most critical operations under a defined workload. The results, obtained in different working conditions and with different infrastructure configurations, are presented and discussed to draw the main considerations about the general problem of defining boundary, granularity, and optimal placement of microservices on the underlying execution environment. In particular, they show that knowing how a specific workload impacts the constituent microservices of an application, helps achieve better performance, by effectively lowering response time (e.g., up to a 61% reduction), through the exploitation of locality-driven clustering strategies for deploying groups of services.

KEYWORDS

clustering techniques, distributed tracing, locality, microservices, performance

Abbreviations: ARC, Assignment based on reaching centrality; DBMS, Database management system; DAG, Directed acyclic graph; F, Fast net; GUI, Graphic user interface; IQR, Interquartile range; LB, Locality bounded; LA, Locality average; NFs, Network functions; NUMA, Not uniform access memory; OTLP, Open telemetry protocol; PDS, Parallel and distributed simulations; QoS, Quality of service; RKE, Rancher kubernetes engine; S, Slow net; SP, Spectral; SUT, System under test; TAT, Turn-around time; VM, Virtual machine; WL-A Model, Workload application model.

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2024 The Authors. *Software: Practice and Experience* published by John Wiley & Sons Ltd.

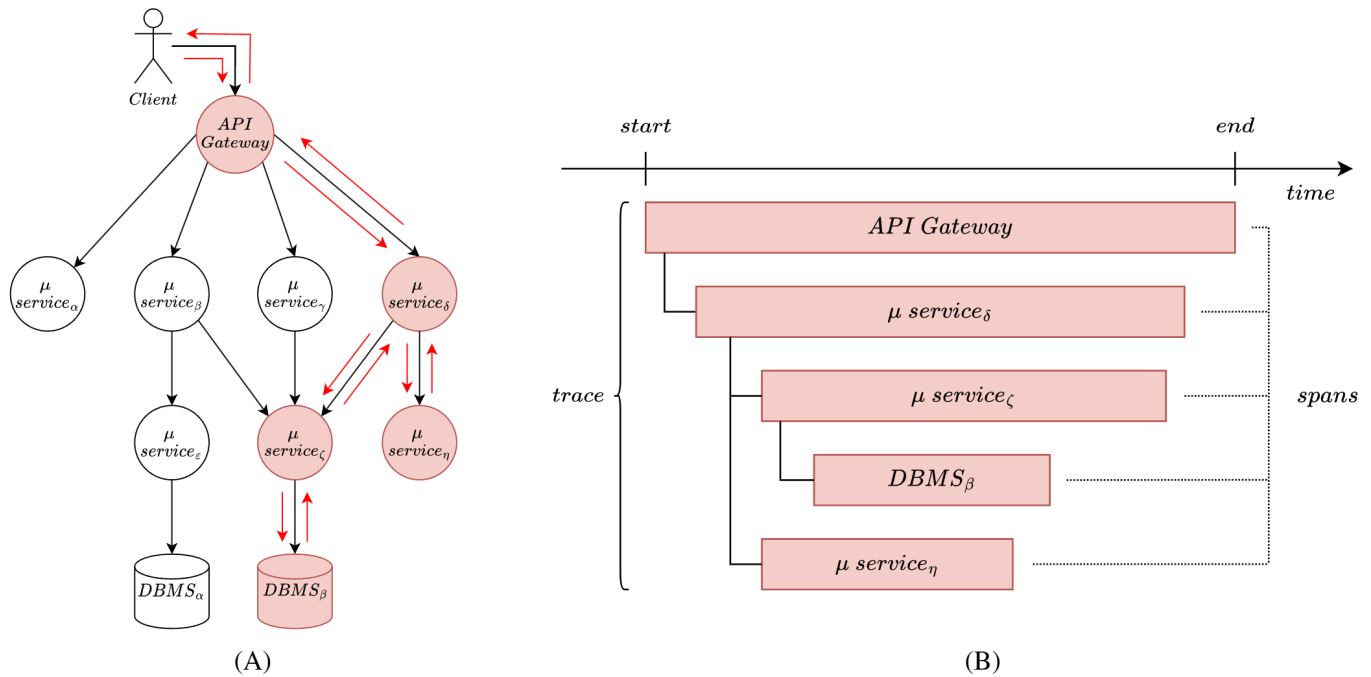


FIGURE 1 Example of trace and spans. (A) Interactions among microservices. (B) Trace report.

1 | INTRODUCTION

The microservices architectural style¹ promotes the development of applications through the composition of independently deployable units.² They are multi-service applications where constituent services are owned and managed by the same authority (tenant). The style and the related agile development approaches improve scalability³ and maintenance, but a bad placement of the microservices onto the underlying (physical or virtualized) hardware could lead to increased response times and performance degradation due to excessive inter-machine communication.

Locality-driven placement can reduce network latency, and consequently response times, by co-locating the most interacting microservices. Several research efforts have been carried out with this aim in distributed applications, as extensively reported in Section 2. In recent years, distributed tracing⁴ has been emerging as a promising technique for understanding the flow of requests and responses between microservices, reducing the time and effort required to diagnose and resolve issues, as well as providing valuable insights into how the system is behaving under different conditions. By tracing requests as they flow through the system, interaction patterns, response times, delays, or bottlenecks can be observed to make informed decisions about how to place and configure microservices with the aim of optimizing performance. The large amount of knowledge collected with distributed tracing enables architects and developers to improve the overall resilience and scalability of the analyzed system.

The main concepts of distributed tracing are *trace* and *span*, whose graphical representations are present in Figure 1. A *trace* is a directed acyclic graph (DAG) of *spans*, which are generated for each invocation of operation during the application execution. Each span contains several (*key*, *value*) pairs referring to different attributes, such as *service name*, *operation name* and *span ID*, *parent span ID*, *start* and *end* timestamps. Instrumentation is supported by specialized libraries or frameworks that provide hooks and middleware to automatically capture and propagate trace contexts.

The most commonly used technologies^{5,6} are OPENTELEMETRY,^{*} JAEGER,[†] and ZIPKIN.[‡] The collected traces can be processed to obtain a graph model for representing the relationships and dependencies between the constituent microservices of an application and to identify potential areas of complexity or risk, for example, tightly coupled microservices

^{*}<https://opentelemetry.io/>.

[†]<https://www.jaegertracing.io/>.

[‡]<https://zipkin.io/>.

or potential single points of failure. Up to now, distributed tracing has been applied to microservices-based systems for different goals: *visualization*,⁷ *performance analysis*,^{8,9} *fault diagnosis*,¹⁰ *anomaly* or *smell detection*.^{11–13}

In this paper, we exploit distributed tracing to define a big data pipeline to dynamically collect tracing data from running applications that are used to identify a given number k of microservices groups that allow for keeping low the response times of the most critical operations under a defined workload, when they are deployed onto different virtual or physical machines, for example, during DevOps stages.

The deployment proposals, obtained by applying clustering-based locality-aware strategies in different working conditions and with different infrastructure configurations, are presented and discussed to draw the main considerations about the general problem of defining the granularity of microservices and their optimal placement on the underlying execution environment. In this work, we do not consider sizing constraints for the execution environment, since they can be easily applied to the proposed clustering techniques to identify more constrained configurations.

The main and novel contributions of the work are:

1. the definition, the construction by tracing, and the adoption of a graph-based runtime model of the application, called *Workload Application Model* (WL-A Model), that synthesizes the interaction patterns between microservices, with their load intensity, obtained by stimulating the application with an actual workload, and that we exploit to find optimal mappings (focusing on locality) of the microservices themselves on the execution environments via clustering techniques. Such a model can be exploited to perform dynamic placements, leveraging the information associated with the actual workload distribution across microservices. This capability allows us to optimize the allocation of the latter thus enhancing overall system performance in response to ever-changing clients' requests;
2. the study of the impact of scalability on the preservation of locality benefits when the workload increases. We show that the granularity changes, due to the growing service time when load increases drastically, reduce the positive effects of locality on response times if the system under test is not adequately scaled.

As a further contribution, we experimented with spectral clustering to address the problem of microservices locality, which, to the best of our knowledge, has never been employed in this sense.

The rest of the paper is organized as follows: in Section 2, we analyze the state of the art to position our work; in Sections 3 and 4, we present the proposed framework along with its application to a real case study; in Section 5, we describe the different steps performed to validate our proposal by also introducing different evaluation metrics, whereas in Section 6, we discuss the obtained results; finally, in Section 7, we conclude the paper and highlight future research directions.

2 | RELATED WORK

The problem of properly placing microservices in the Cloud to improve performance by reducing response times has been addressed in different ways and several of them try to exploit locality. The following literature review does not consider edge/fog infrastructures, since the main scope of this paper is Cloud-native applications based on microservices.

A first group of works is based on heuristics. Han et al.,¹⁴ define a refinement framework that exploits both workload profiling and monitoring data to (i) dynamically select the most suitable cluster among a predefined set of Kubernetes clusters where to deploy all the microservices, and (ii) place these latter in such a way as to minimize the interactions and data exchanged among the nodes of the cluster. Zhao et al.¹⁵ define a scheduling strategy for containerized cloud services that considers resource usage metrics (CPU, disk, memory utilization, and I/O bandwidth) to find a deployment that reduces network traffic and at the same time local I/O contentions. REMaP⁶ is a MAPE-K-based adaptation manager designed to dynamically handle microservices placement based on their reciprocal affinities, which depend on the number and size of messages exchanged over time, and resource usage history (e.g., CPU, memory, and disk utilization): microservices with high affinity should be placed together in order to reduce communication latency, but at the same time, microservices with a history of high resource usage should not be co-located. The planning component sorts pairs of microservices in decreasing order of affinity and then tries to place each pair on the same host. The authors also propose a planner that makes use of a SAT solver to find the optimal placement, which is the one that minimizes the number of hosts and maximizes the affinity scores on each of them. MicroRanker¹⁶ is a placement service that ranks microservices

based on the number of their interactions in terms of mutual calls, and then pools them together according to the ranking produced and by taking into account the available number of hosts. Each pool is subsequently deployed on a specific host that is selected considering its green index. The goal is to minimize the network traffic and increase the utilization of green energy among geographically distributed hosts. Hu et al.¹⁷ propose a placement approach that minimizes the network traffic among the machines hosting the service-based applications, satisfying the latter's resource and traffic demands. The envisioned solution first partitions the application by keeping the overall traffic between the different parts to the minimum and then packs each partition into a machine while respecting resource constraints. IntMA⁵ is a microservices allocation mechanism that minimizes interaction costs by complying with the resource constraints of the nodes that make up the execution environment. The proposed solution works on the service interaction graph, and it is inspired by the minimum spanning tree solution: it tries to place as many strictly dependent microservices as possible on the same node.

A second group of works leverages genetic algorithms. This kind of approach evaluates a set of potential solutions through a fitness function; the best solutions are then evolved to create new candidates, which are in turn evaluated. This process is repeated iteratively until a certain condition is met (e.g., the fitness function has reached a predefined value or a given number of generations have been produced). Ding et al.¹⁸ propose Kubernetes-oriented microservice placement method that determines the optimal number of instances per microservice while minimizing both resources and communication costs under the following constraints: (i) the response time must be kept under a given threshold, (ii) the total resource demand can exceed the total amount of resource available in presence of multiple instances of the same microservice, and (iii) the shared dependency libraries between the diverse instances cannot exceed the storage capacity of nodes hosting the whole application. HPA¹⁹ combines genetic algorithms and spectral clustering to find a microservices placement in multi-cloud environments that combines two stages to minimize the so-called Turn-Around Time (TAT), that is, the sum of the initialization time, the function processing time, and the communication time. In the first stage, microservices are mapped to clusters, whereas in the second stage, they are mapped to the specific cluster nodes.

The placement problem has also been faced in other contexts, such as network function virtualization and distributed scientific applications. For example, Invenio²⁰ is a system for computing communication affinity among network functions (NFs), with the aim of using this information to properly place NFs, specifically by co-locating functions with high affinity in order to minimize latency drawbacks. Zaheer et al.²¹ instead, propose a placement algorithm for Parallel and Distributed Simulations (PDS) able to minimize the communication costs in terms of hop count, where the cost of each communication is obtained by multiplying the number of traversed hops by the number of exchanged messages. The procedure iteratively assigns the highest-message-exchanging pair of processes to the racks until their resource limits are reached.

All the works above address the microservices placement problem by proposing only off-line algorithms, exploiting various kinds of data, that do not consider the need for running and finding solutions in the deployment stage of a DevOps pipeline. To the best of our knowledge, only MicroRanker¹⁶ and REMaP⁶ take care of this aspect. While the former mainly focuses on reducing energy consumption and carbon emissions in a multi-data center scenario (by giving a few details about microservices interaction monitoring), the latter addresses the problem of reducing response times, as in our case. The authors propose two allocation algorithms that work at runtime to reconfigure the placement by moving microservices on the basis of the collected data. This approach forces the adoption of instrumented microservices also in production with significant performance degradation, as the same authors claim. The approach we propose, on the contrary, does not need continuous monitoring, since we separate locality optimization from resource scaling and load balancing (that we do not address in this paper). Therefore, we only need to analyze the impact of the behaviors present in a given workload onto the application to discover strong dependencies between services that suggest to co-locate them with the aim of obtaining an efficient mapping of the WL-A model onto the underlying hardware. Moreover, REMaP has been tested only with a simple application composed of a small number of microservices where coupling mainly regards the business logic with the related databases and the impact of locality induced by the proposed allocation algorithms remains unclear. On the other hand, the works that mention distributed tracing^{5,6} do not use it systematically, nor as part of a big data processing pipeline; a reference to distributed tracing is made to describe the types of data used by the various methods and techniques as a source for planning. Finally, only HPA¹⁹ provides for the use of spectral clustering that, differently from our approach, is employed only for scaling purposes, specifically to reduce the number of hosting environments by grouping them based on their similarity in terms of communication capabilities.

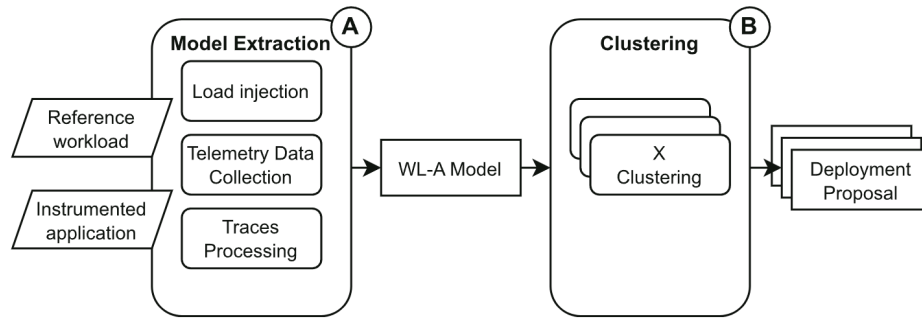


FIGURE 2 Framework overview.

3 | FRAMEWORK FOR MICROSERVICES PLACEMENT

In this section, we introduce the conceptual framework we exploit to generate locality-aware placements of microservices, whose working steps are resumed in Figure 2 and detailed and discussed in the next subsections. The framework is composed of two steps: (a) model extraction and (b) clustering.

Step ① involves the collection and processing of execution traces with the aim of building a graph-based runtime model that can be exploited to study and analyze the application. We then use it as input of step ② to produce deployment proposals, via clustering techniques, that keep response times low by exploiting locality.

In the following sections, we describe the two steps in more detail, and we present the formal definition of the envisioned model.

3.1 | Model extraction

This phase aims to build a run-time model of the application under test, taking into account the specific workload. In fact, the model is influenced both by the application structure and by the set of invocations (with related parameters) performed by users on it. In the following, we use the term “reference” to identify this workload, and the related run-time model is named *Workload-Application* (WL-A) model.

The workload can be defined as follows: (i) if the system is in production, it can be directly derived from execution logs; (ii) if the system is not in production yet, it can be created by considering a typical usage that tries to cover most of the operations exposed by the microservices composing the application under test.

Once defined, the workload is injected into the system. Since an application may be composed of a large number of microservices, as well as a high number of users, the telemetry subsystem must be prepared to gather all spans without losing information (due to the number of spans or the rate at which they are produced). To this end, a monitoring toolchain is created to extract and store traces and metrics using OTLP (OPEN TELEMETRY PROTOCOL), as described in Figure 3.

The toolchain is a big data collection pipeline designed to manage the large amount of information produced during workload injection. The pipeline’s main components are: (i) the agent, placed alongside the execution environment of application microservices, and responsible for gathering traces and metrics, and sending them to a specific collector; (ii) the collectors, responsible for collecting trace and metrics data from distributed agents and forwarding them to a stream broker; (iii) the stream broker, responsible for quickly absorbing the big amount of data forwarded by the collector and storing them before further processing (e.g., data conversion, online analysis); (iv) the persistence sub-system, responsible for storing the ingested data.

The *OTLP agent* monitors the application environment and produces spans when its instrumented procedures are stimulated by the workload. The spans are then grouped in batches and forwarded to an *OTLP collector*. This is composed of two sub-components, namely *OTLP receiver* and *Kafka exporter*: the former receives spans aggregated in batch and represented in *protobuf* data format, whereas the latter sends them to a *Stream broker*. The broker is an *APACHE KAFKA*[§] cluster that is able to handle and store large amounts of data with high-throughput and low-latency.

[§]<https://kafka.apache.org/>.

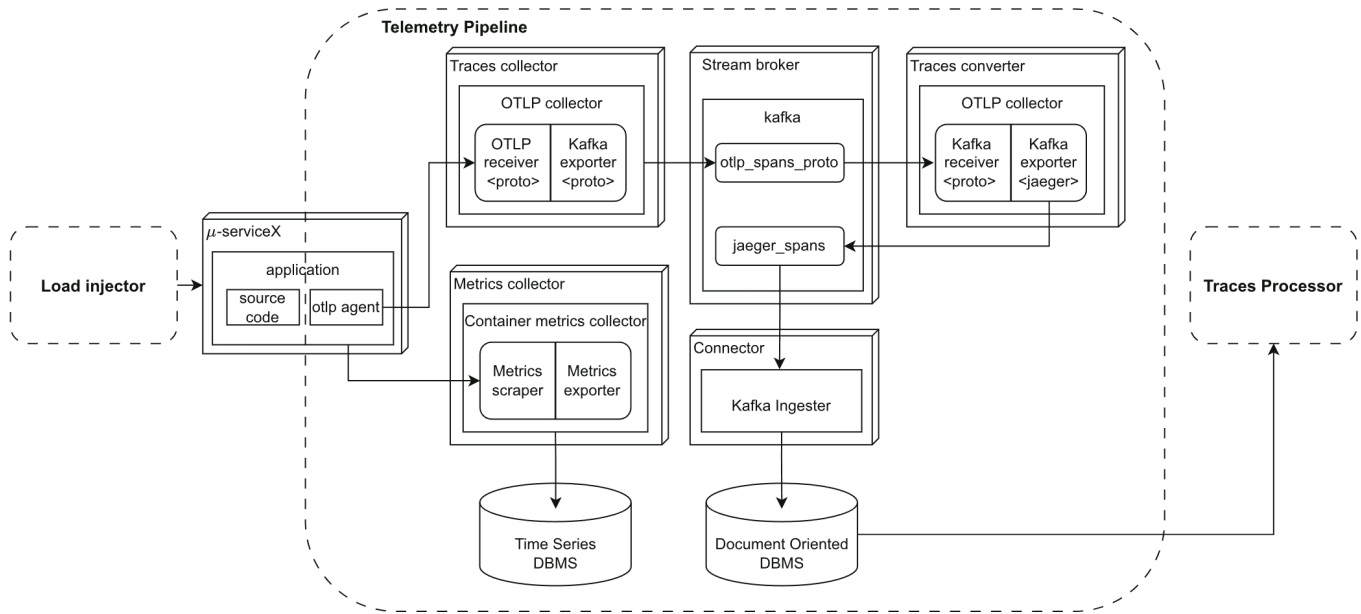


FIGURE 3 Telemetry collection and processing components.

The *Kafka exporter* transmits the aggregated spans to a topic called “*otlp_spans_proto*”, encapsulating them inside messages whose payload is encoded in “*otlp_proto*”. Using this encoding, together with the protobuf data format, allows us to quickly collect the spans, minimizing the ingestion latency, but unfortunately prevents us from processing them. Consequently, we introduce an additional *OTLP collector* that acts as a data format converter and that is composed of a *Kafka receiver*, which is responsible for polling data from the “*otlp_spans_proto*” topic, and a *Kafka exporter*, which converts the spans read and saves them into the “*jaeger_spans*” topic using the “*jaeger_json*” encoding. Although this step could be performed directly by the first collector, we avoid this since it could introduce overheads that would negatively impact the ingestion phase, leading for example to corrupted or even lost spans. Data converted are finally persisted to a *Document Oriented DBMS*, in our case a MONGODB[¶] instance, via a sink connector (*Kafka ingestor*).

Parallel to this first path, we have a second path related to the ingestion of metrics data, that we subsequently use for further investigation during our analysis. In particular, metrics are collected via a *Container metrics collector* that gathers data from the probes directly installed inside the execution environments hosting the microservices (typically container-based environments). Metrics are then persisted to a *Time Series DBMS* implemented with PROMETHEUS.[#]

3.2 | WL-A model

Telemetry data composing execution traces and stored in the persistence layer is used to construct a workload-dependent graph-based runtime model of the application.

Definition 1. A WL-A model is a multi-layer directed property graph $G(V, E, D_V, D_E, \{\lambda_V^i\}_{i=0}^w, \{\lambda_E^j\}_{j=0}^z)$, where V is the set of n nodes, E is the set of m edges, D_V is the set of node layers, each one referring to a specific type of node, D_E is the set of edge layers, each one referring to a specific type of edge, and $\{\lambda_V^i\}_{i=0}^w$ and $\{\lambda_E^j\}_{j=0}^z$ are two sets of functions that associate nodes and edges with properties and assign them specific values.

Layers refer to disjoint groups of entities (nodes or edges) and can be used to build different views of the graph. Properties represent both static and dynamic attributes that characterize the entities and that are meaningful for the purposes of the subsequent analysis. Each function introduced in the model refers to a distinct property. In particular, functions are defined as follows.

[¶]<https://www.mongodb.com/>.

[#]<https://prometheus.io/>.

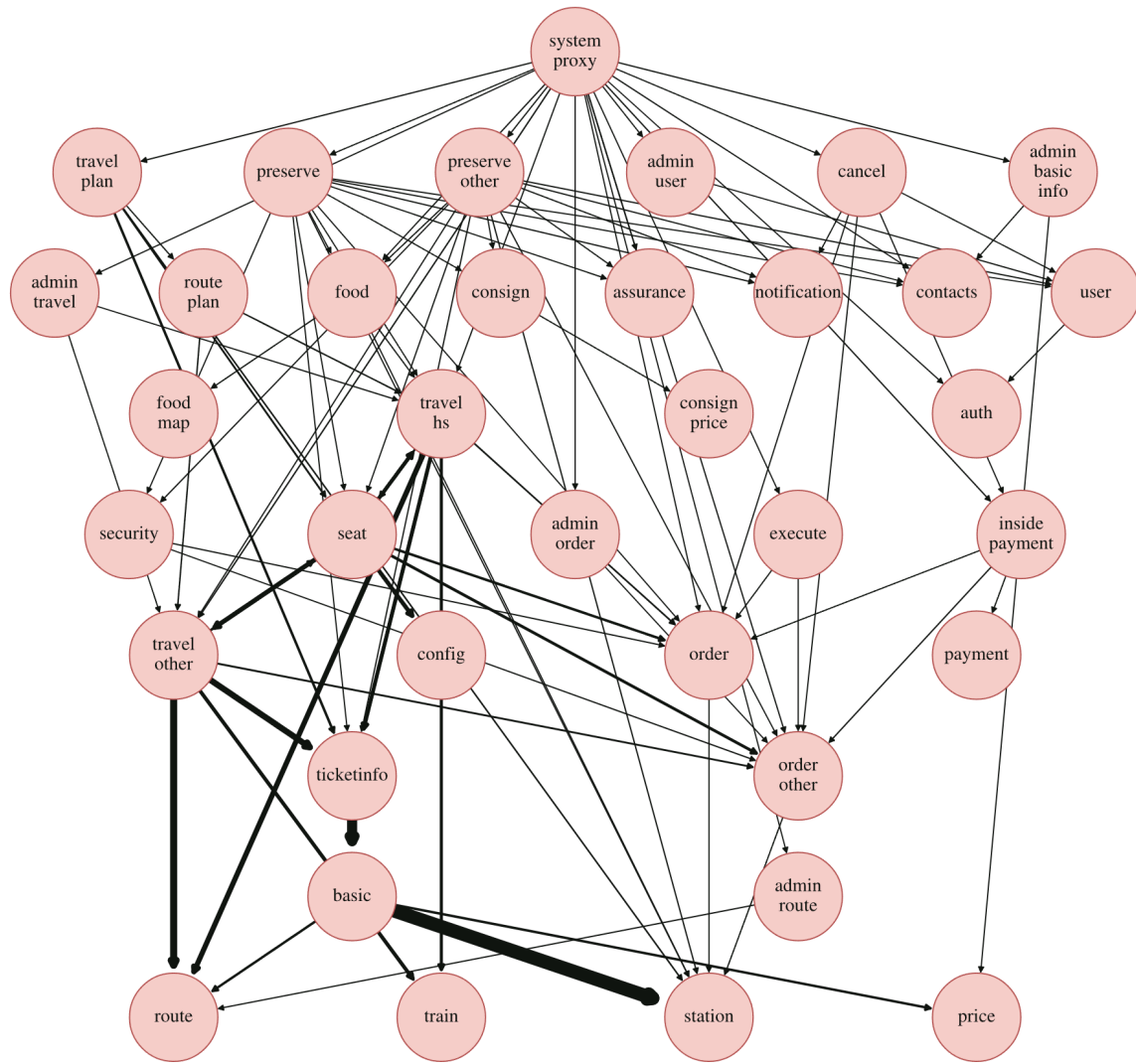


FIGURE 4 Example of the WL-A model microservice layer for TrainTicket application.

Definition 2. Given a node (edge) property p_V^i (p_E^j) with its corresponding set of admissible atomic or complex values P_V^i (P_E^j), a function λ_V^i (λ_E^j) is a one-to-one-relationship $\lambda_V^i : V \times \{p_V^i\} \rightarrow P_V^i$ ($\lambda_E^j : E \times \{p_E^j\} \rightarrow P_E^j$).

We use p to indicate a property and P to indicate the set of its admissible values. From a semantic point of view, nodes represent the *operations* invoked at runtime, the latter referred to as *OPs*, while edges represent *calls* between pairs of operations.

An operation can refer to both exposed or internal microservices endpoints, that is, endpoints reachable by clients or endpoints that only serve requests coming from other microservices that are part of the same application, and to internal methods or functions (according to the programming paradigm adopted). This aspect is captured by exploiting the concept of layer introduced in our model. For example, we can set $D_V = \{MICROSERVICE, ENDPOINT, METHOD\}$ to take into account the different kinds of entities, or we can limit the scope of the analysis to only one of them, as in the case of this work, in which we only consider the layer of the microservices by setting $D_V = \{MICROSERVICE\}$, as reported in the example of Figure 4. In addition, we set $D_E = \{CALL\}$, since we are interested in keeping track of the interactions that occur between the microservices.

We use properties to host some metadata useful to uniquely identify the microservices and the telemetry data we then exploit to define the microservices placement. In particular, we introduce the following functions:

$$\lambda_V^0 : V \times \{p_V^0 = \text{"uid"}\} \rightarrow P_V^0 = \mathbb{N} \quad (1)$$

$$\lambda_V^1 : V \times \{p_V^1 = \text{"uname"}\} \rightarrow P_V^1 = \{[A - Z a - z 0 - 9 _ -] + \} \quad (2)$$

$$\lambda_E^0 : E \times \{p_E^0 = \text{"weight"}\} \rightarrow P_E^0 = \mathbb{R}_+, \quad (3)$$

where *uid* and *uname* are a unique integer identifier and name, respectively, and *weight* is a numeric value that synthesizes the metric exploited by clustering. The proposed model allows us to easily capture the changes that occur dynamically in the application structure and/or in the interaction patterns between its constituent parts as a consequence of the workload modifications. For example, an increase in communication intensity between two microservices is reflected in the model as an increase in the weight on the edge that links the nodes representing the microservices themselves.

3.3 | Clustering

In step ③, we apply clustering techniques to produce multiple labeling of the nodes of the modeling graph that represent the cluster membership relationships. Each labeling then translates into a different deployment proposal.

Labeling performed applying a given clustering technique X consists of introducing a new function $\lambda_V^i = V \times \{p_V^i = \text{"cluster } X"\} \rightarrow P_V^i = \mathbb{N}$, that is, a new property with its admitted values, assuming we refer to an individual cluster with a numeric identifier. Each group of nodes represents a set of microservices that should be deployed together within the same execution environment (e.g., a virtual machine). Regardless of the specific kind of technique being used, which can be a custom one or any of those proposed in the literature (see Section 2), the rationale behind grouping nodes should favor locality, that is, the most interacting microservices should be placed together, in order to reduce communication overhead and achieve better performance. To this end, every technique leverages the metric values stored in the *weight* property associated with the edges of the modeling graph (see Section 3.2), interpreting them as *affinity* intensities: the higher the value, the more likely it is that the microservices involved are placed together.

The data from which the metric is derived depends on the nature of the application under analysis. For example, if the latter is *computationally intensive*, then the resource demands (CPU and memory) should be taken into account; if it is *data intensive*, then the amount of data that microservices exchange with each other should be considered; if instead, it is *network intensive*, then the number of invocations would be a good indicator to select related microservices; if it is of multiple types, then a combination of the indicators should be used. In terms of constraints, we only impose a fixed number of final clusters, since, in this first proposal, we assume that the execution environment infrastructure hosting the application already exists and it has a predefined sizing.

4 | CASE STUDY

In this section, we describe the application of the proposed framework to a case study.

4.1 | Sample application

To test the framework, we choose *TrainTicket*,^{||} an open-source microservices-based application for Chinese train ticket booking system, which is often used as a benchmark application. In particular, we used the 0.0.4 release, composed of 41 microservices and 22 databases, all of them individually encapsulated inside a dedicated container. Specifically, it uses 20 different instances of MongoDB DBMS v-5.0 and 1 instance of MySQL v-5.6.35. The microservices are implemented and hosted as follows: 1 PYTHON v-3.8 container, 1 NODE.JS v-14.5 container, 1 GO v-1.6.3 container, 1 NGINX OPENRESTY

^{||}<https://github.com/FudanSELab/train-ticket>.

container, and 38 JAVA-8 containers with Spring application framework v-1.5.22 installed inside them. From now on, we often refer to this application as System Under Test (SUT).

The SUT needs some interventions to enable the production of telemetry data. Specifically, we attached an *OTLP Java agent* JAR to all the Java microservices in order to observe them and send data to collectors (see Section 3.1). Moreover, to ensure compatibility with the agent, we (i) upgraded the Java version used inside the containers to v-11 and the Spring version to v-2.3.12, and (ii) replaced the original NGINX front-end with a custom one implemented with SPRING that worked as a proxy. The latter has been designed to keep track of the microservice interactions starting from the system entry-point and throughout the chains of calls.

Finally, we disabled the *circuit breaker* provided by the *Netflix Hystrix* module to avoid admission control that could masquerade high response times. In particular, this prevents operations from being interrupted due to timeouts and therefore allows us to monitor the actual duration of the interactions.

4.2 | Reference workload

The workload used to stimulate the selected microservices-based application was derived by considering all the external endpoints, that is, those that can be accessed by clients. Table 1 summarizes them by reporting their corresponding URLs and HTTP methods, along with their unique identifiers and the microservices they refer to. In general, the aim is to cover as many microservices and paths of the SUT as possible.

The analysis of the operations provided by the application GUI suggested grouping them by type of users (actor). Moreover, taking into consideration the pre- and post-conditions of each operation, we created 28 admissible operation sequences. Each of them is an admissible behavior for a specific actor.

According to the characteristics of the application and the expected user behaviors, we defined a realistic operation frequency distribution, reported in Table 2. Specifically, we identified five different kinds of actors who represent *logged* or *external* (not logged) users, users who are interested in high-speed (*hs*) or normal (*other*) trains, and users with admin roles. Each different actor is characterized by a probability of being selected and a probability of performing one of the behaviors (b_i) during load injection. For example, an actor of type external-other (a user who is not logged in and is interested in non-fast trains), has a probability of 48% of being selected, as opposed to an actor of type admin who has a probability of 5%. Each actor, in turn, has his own probability of performing a behavior, that is, a sequence of operations. As an example, for an actor of type external-other, one of the $b_5 - b_8$ sequences can be selected, with a ratio of 1 to 2 between b_5 and the remaining ones. In general, the actor probability equals the sum of its sequence probability. Values have been chosen to simulate a realistic workload based on the type of actor. For example, for a logged user it is reasonable to think that the invocation of the *delete* behavior (b_{12} , b_{18}) is significantly lower than the *search and preserve* ones (b_{10} , b_{11} , and b_{16} , b_{17}).

A given actor can follow one or more behaviors within the same session, but every session starts from the actor's first sequence, which is the initialization one. For example, the logged-hs actor must begin his session with the b_9 setup sequence in order to be able to perform any other sequence ($b_{10} - b_{14}$).

4.3 | Underlying infrastructure

In this section, we describe the physical end virtual environment used to conduct the experimental evaluation of the framework.

4.3.1 | Physical machines

To conduct our experiments, we exploited a highly scalable system^{**} composed of High-Performance NUMA machine managed by a controller machine. The NUMA machine is equipped with eight Intel Xeon Platinum 8360H @3.00GHz CPU base clock frequency (384 virtual cores), 2TB of RAM, and 13.8TB of SAS SSD. The controller machine is equipped

^{**}A cluster based on two Superdome 280 Flex chassis hosted by RCOST (Research Center On Software Technology) at University of Sannio—Department of Engineering.

TABLE 1 Microservices' REST endpoint.

ID	Microservice	Method	Endpoint
ab_1	admin-basic	GET	/adminbasicservice/adminbasic/contacts
ab_2	admin-basic	GET	/adminbasicservice/adminbasic/prices
ab_3	admin-basic	PUT	/adminbasicservice/adminbasic/contacts
ab_4	admin-basic	PUT	/adminbasicservice/adminbasic/prices
ao_1	admin-order	POST	/adminorderservice/adminorder
ao_2	admin-order	GET	/adminorderservice/adminorder
ao_3	admin-order	PUT	/adminorderservice/adminorder
ar_1	admin-route	GET	/adminrouteservice/adminroute
at_1	admin-travel	POST	/admintravelservice/admintravel
at_2	admin-travel	GET	/admintravelservice/admintravel
at_3	admin-travel	PUT	/admintravelservice/admintravel
au_1	admin-user	POST	/adminuserservice/users
au_2	admin-user	GET	/adminuserservice/users
as_1	assurance	GET	/assuranceservice/assurances/types
ca_1	cancel	GET	/cancelservice/cancel/{orderId}/{userId}
ca_2	cancel	GET	/cancelservice/cancel/refound/{orderId}
co_1	contacts	GET	/contactservice/contacts/account/{userId}
da_1	dashboard	GET	/admin.html
da_2	dashboard	GET	/client_login.html
da_3	dashboard	GET	/index.html
ex_1	execute	GET	/executeservice/execute/collected/{orderId}
ex_2	execute	GET	/executeservice/execute/execute/{orderId}
fo_1	food	GET	/foodservice/foods/{date}/{s1}/{s2}/{tripId}
ip_1	inside-payment	POST	/inside_pay_service/inside_payment
oh_1	order-hs	POST	/orderservice/order/refresh
oo_1	order-other	POST	/orderOtherService/orderOther/refresh
ph_1	preserve-hs	POST	/preserveservice/preserve
po_1	preserve-other	POST	/preserveotherservice/preserveOther
th_1	travel-hs	POST	/travelservice/trips/left
to_1	travel-other	POST	/travel2service/trips/left
tp_1	travel-plan	POST	/travelplanservice/travelPlan/cheapest
us_1	user	POST	/users/login

with an Intel Xeon Silver 4208 CPU @2.10GHz base clock frequency (16 virtual cores), 64GB of RAM, and 2TB of SATA hard disk at 7.2k rpm.

4.3.2 | Virtualization layers

The IaaS software OPENSTACK is used to build a cluster of eight VMs that is installed on top of the physical machines. These VMs are configured to host a Kubernetes cluster, implemented with RANCHER KUBERNETES ENGINE (formally

TABLE 2 Reference workload characterization.

Actor	Actor probability	Sequence ID	Operations	Sequence probability	Description
External-hs	12%	b_1	da_3	1.71%	Home
		b_2	th_1	3.43%	Search travel hs oneway
		b_3	th_1, th_1	3.43%	Search travel hs roundtrip
		b_4	tp_1	3.43%	Travel plan
External-other	48%	b_5	da_3	6.86%	Home
		b_6	to_1	13.71%	Search travel other oneway
		b_7	to_1, to_1	13.71%	Search travel other roundtrip
		b_8	tp_1	13.71%	Travel plan
Logged-hs	7%	b_9	da_3, da_2, us_1, da_3	0.90%	Home and login
		b_{10}	$th_1, as_1, fo_1, co_1, ph_1, ip_1$	1.80%	Search and preserve hs oneway
		b_{11}	$th_1, as_1, fo_1, co_1, ph_1, ip_1, th_1, as_1, fo_1, co_1, ph_1, ip_1$	1.80%	Search and preserve hs roundtrip
		b_{12}	oh_1, ca_2, ca_1	0.36%	Delete ticket
		b_{13}	tp_1	1.80%	Travel plan
		b_{14}	$oh_1, oo_1, ex_1, oh_1, oo_1, ex_2$	0.36%	Execute
Logged-other	28%	b_{15}	da_3, da_2, us_1, da_3	3.60%	Home and login
		b_{16}	$to_1, as_1, fo_1, co_1, po_1, ip_1$	7.18%	Search and preserve hs oneway
		b_{17}	$to_1, as_1, fo_1, co_1, po_1, ip_1, to_1, as_1, fo_1, co_1, po_1, ip_1$	7.18%	Search and preserve hs roundtrip
		b_{18}	oh_1, ca_2, ca_1	1.43%	Delete ticket
		b_{19}	tp_1	7.18%	Travel plan
		b_{20}	$oh_1, oo_1, ex_1, oh_1, oo_1, ex_2$	1.43%	Execute
Admin	5%	b_{21}	da_1, us_1, da_1, ao_2	0.34%	Home and login
		b_{22}	ar_1, at_2, at_1	0.67%	Create travel
		b_{23}	at_2, at_3	0.67%	Modify travel
		b_{24}	ao_3	0.67%	Update order
		b_{25}	ao_1	0.67%	Create order
		b_{26}	ab_2, ab_4	0.67%	Modify price
		b_{27}	ab_1, ab_3	0.67%	Modify contact
		b_{28}	au_2, au_1	0.67%	Add user

RKE).^{††} Specifically, we have five VM equipped with 12 vCPUs, 32 GB of RAM and 100GB of disk storage (*Small* flavor); one additional with 48 vCPUs, 120 GB of RAM and 400GB of disk storage (*Big* flavor); and another one with 36 vCPUs, 90 GB of RAM and 300GB of disk storage (*Intermediate* flavor). One of the 12-vCPUs VM is used as Kubernetes master.

Figure 5 resumes VMs sizing and placement on corresponding physical machines; the colors represent the VM role in our testing environment:

1. *Blue*: machines used for system management. The Master (*Small* flavor) manages the containers' lifecycle and the network overlay used to communicate with each other. We do not deploy other services on this machine to prevent control interference on the application;

^{††}<https://www.rancher.com/products/rke>.

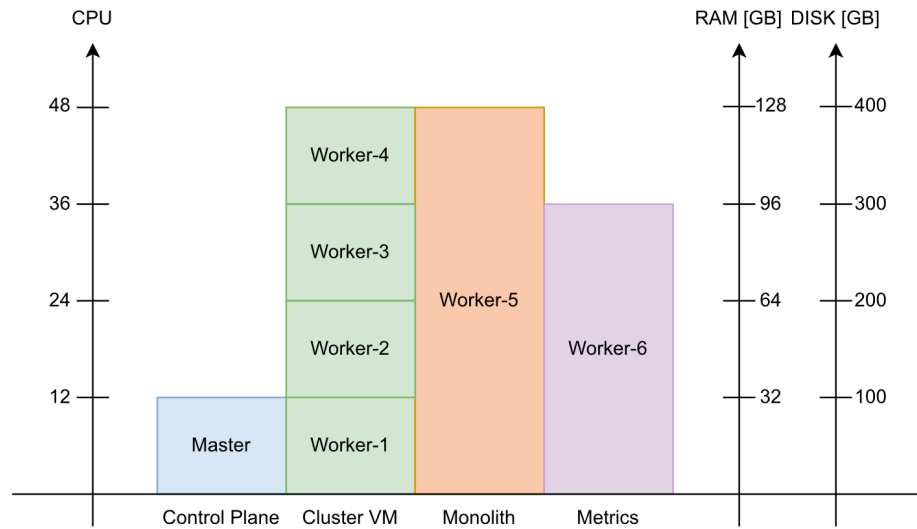


FIGURE 5 Virtual machines sizing.

2. *Green*: machines used to deploy the application to test. Worker-(1, 2, 3, 4) belong to this group. They have a *Small* flavor and communicate with each other via a virtualized network that resides on the same physical machine;
3. *Orange*: machines used to deploy the application to test, in the absence of resource partitioning and network (services will use a local addressing space). Only the *Big* flavored Worker-5 belongs to this group. Note that its resources are approximately equal to the sum of the ones of the machines in the green group;
4. *Purple*: machines used for telemetry data collection. Worker-6 (*Intermediate* flavor), hosts the proposed collection and processing toolchain (Figure 3);

4.3.3 | Networks

VMs communicate with each other through a high-performance network, upon which we have performed latency and bandwidth control operations to assess our framework under two different communication setups:

1. Fast net (**F**), whose latency is similar to the one observed for intra-computer cluster communication. We rely on traffic control (*tcconfig*^{††}) utility program to set a latency of 200 μ s for VM incoming and outgoing network traffic, with a consequent round trip latency of 400 μ s;
2. Slow net (**S**), whose latency is similar to the one observed for inter (computer) cluster communication. With the same utility, we set a latency of 2 ms and a round trip latency of 4 ms.

4.4 | WL-A model extraction

The extraction of the WL-A model requires the deployment of the SUT. To this end, each SUT container is encapsulated in a Kubernetes Pod and deployed into our previously described RKE-managed cluster. All Pods are deployed into a single Worker machine (the Orange colored Worker-5 of Section 4.3) since we are only interested in interaction patterns. Once deployed, the SUT is subjected to the reference workload (see Table 2), which is injected through the “*Locust*”^{§§} open-source load testing tool. The spans are stored in a DBMS, from which they are subsequently retrieved to be processed in order to generate the WL-A Model. To this end, they are first linked to each other to reconstruct the DAGs modeling the execution traces to which they refer; then, these tree-structured graphs are merged together in order to build a single final graph. Each DAG refers to an actual chain of invocations; this means that, at the first level, the nodes represent

^{††}<https://tcconfig.readthedocs.io/en/latest/index.html>.

^{§§}<https://locust.io/>.

both instances of operations of any kind (microservice endpoints and internal methods). Consequently, to obtain a final graph compliant with the model presented in Section 3.2 we aggregate nodes by the “*service_name*” field that is directly exposed by the telemetry probes and that we use to populate the *uname* property (see Figure 4). The *uid* property hosts a numerical identifier that is incremented every time a new microservice is encountered. Finally, the *weight* property is initialized to 1 when a new link is discovered and updated whenever an existing link is found.

4.5 | Clustering techniques

In the following, we present the different clustering techniques we experimented with to suggest locality-aware deployment of the microservices onto a given number k of predefined VMs, each one equipped with a sufficient amount of computing resources. All the techniques implemented take as input a directed weighted graph G , which corresponds to the WL-A graph model, whose weights on the edges are the min-max normalized number of direct calls between pairs of microservices.

Each technique produces a node labeling that represents the cluster membership relationships. Consequently, in our model, we introduce three functions (see Equations (4), (5), and (6)), one for each technique, where the suffixes of the properties names are the labels we will use in the rest of the paper to refer to the diverse methods, which are described below.

$$\lambda_V^2 : V \times \{p_V^2 = \text{“cluster}_{SP}”\} \rightarrow P_V^2 = \mathbb{N} \quad (4)$$

$$\lambda_V^3 : V \times \{p_V^3 = \text{“cluster}_{ARC}”\} \rightarrow P_V^3 = \mathbb{N} \quad (5)$$

$$\lambda_V^4 : V \times \{p_V^4 = \text{“cluster}_{LB}”\} \rightarrow P_V^4 = \mathbb{N} \quad (6)$$

4.5.1 | Spectral

This technique has been first proposed by Pothen et al.,²² and involves applying clustering to a projection in a lower dimensional embedding space of the normalized Laplacian $L = 1 - D^{1/2}AD^{1/2}$, the latter obtained starting from the symmetric weighted adjacency matrix $A \in \mathbb{R}^{n \times n}$ and the weighted degree matrix $D \in \mathbb{R}^{n \times n}$, both associated with the modeling graph G . Specifically, the first k -eigenvectors of L , which are associated with its first k -eigenvalues sorted in decreasing order of magnitude, form a basis where the nodes are projected in. The idea is to group these nodes, which are represented as vectors of coordinates relative to the new basis, by exploiting one of the classic clustering methods (e.g., k -means). Formally, the k eigenvectors $v_1 = [v_{11}v_{12} \cdots v_{1n}]^T$, $v_2 = [v_{21}v_{22} \cdots v_{2n}]^T$, ..., $v_k = [v_{k1}v_{k2} \cdots v_{kn}]^T$ are concatenated to construct a matrix $M \in \mathbb{R}^{n \times k}$, where the j -th column represents the j -th eigenvector and the i -th row represents the i -th node. Row vectors are typically clustered using k -means. However, this method can be too sensitive to centroid initialization. Therefore, we use the QR method²³ that has no tuning parameters and that may be better in terms of both quality and speed.

4.5.2 | Assignment based on reaching centrality

This technique is a point-assignment-based clustering method that uses the first k nodes with the highest *local reaching centrality*²⁴ value, that is, the percentage of other nodes that can be reached from a given node in a directed network, as initial nodes of the clusters. Each remaining node is then iteratively assigned to the cluster that maximizes the number of interactions with its neighborhood. Formally, let $C = \{C_i\}_{i=0}^k$ be the set of clusters, each one containing an initial node, V_i be the set of nodes that currently form the generic cluster C_i , j be the current node to assign, and $N(j) = \text{Pred}(j) \cup \text{Succ}(j)$ be the neighborhood of j , where $\text{Pred}(j) = \{u : (u, j) \in E\}$ is the set of predecessors and $\text{Succ}(j) = \{u : (j, u) \in E\}$ is the set of successors of j . The cluster i to which node j is assigned is the one that maximizes the following score:

$$s_{ij} = \sum_{u: u \in V_i \cap N(j)} f(u, j) \cdot w_{u,j} + (1 - f(u, j)) \cdot w_{j,u}, \quad (7)$$

where

$$f(u, j) = \begin{cases} 1, & \text{if } u \in \text{Pred}(j) \\ 0, & \text{if } u \in \text{Succ}(j) \end{cases}$$

In the case of ties, we select a cluster at random.

Using local reaching centrality to identify initial nodes helps us to prevent imbalance between clusters (e.g., the formation of a single very large cluster against several very small clusters). This is due to the way the final microservices graph G is built. The latter is indeed obtained by merging the different directed graphs representing the execution traces, which have a tree structure, and it hosts on its edge weights the number of calls between pairs of microservices. Consequently, the most solicited nodes, located closest to root nodes, are those with high values of the metric.

4.5.3 | Locality bounded

This technique is inspired by the work of Zaheer et al.²¹ and aims at maximizing intra-cluster communication by assigning nodes to clusters following an order deriving from the sorting of the edges in decreasing order of weights. Initially, the endpoints of the heaviest edge are assigned to the same cluster. Subsequent nodes are then selected by finding the next heaviest edge, such that one of its endpoints has already been assigned. To avoid the trivial solution consisting of one single cluster, we constrained the maximum capacity of each cluster in terms of the maximum number of microservices that can be grouped together. Formally, given a generic edge e , let e^{src} and e^{dst} be the source and destination endpoints, respectively. Moreover, let $(e_1, e_2, \dots, e_{|E|})$ be the list of edges of G sorted in decreasing order of weight, that is, such that $w_{e_1^{src}, e_1^{dst}} \geq w_{e_2^{src}, e_2^{dst}} \geq \dots \geq w_{e_{|E|}^{src}, e_{|E|}^{dst}}$, C_i be the current cluster being populated, e_j the current heaviest edge, and T the maximum capacity of a generic cluster. At the beginning, we assign e_j^{src} and e_j^{dst} to C_i . Subsequently, we find the next heaviest edge e_n such that $e_n^{src} \in C_i \vee e_n^{dst} \in C_i$. If there is an edge satisfying this condition and C_i has not reached its maximum capacity, we assign e_n^{src} or e_n^{dst} to C_i , whichever has not yet been assigned, and we proceed by finding a new edge. Otherwise, we move to a new cluster and we repeat the steps described above. The entire procedure ends when there are no edges left. In our experiments, we consider a threshold $T = T_{avg}$ equal to the average number of microservices per execution environment rounded up, and for this threshold we use the **LA label**.

5 | DEPLOYMENT PROPOSALS VALIDATION

The outcome of the proposed framework is a set of deployment proposals that we validate for the purpose of evaluating the impact of locality suggested by the clustering algorithms. In particular, we aim to answer the following questions:

1. **Q1.** Does locality-awareness improve performance?
2. **Q2.** Does workload-awareness improve performance?

Locality-awareness refers to the exploitation of a locality-oriented criterion to group microservices, that is, identifying invocation chains that suffer when the related services are deployed in different execution units. Workload-awareness refers instead to the knowledge of the intensity of the interactions taking place between microservices, that is, how many times the invocations between microservices crossing the boundary of different execution units occur at runtime.

From an operational point of view, the validation phase is composed of two steps, as reported in Figure 6. In step ① the SUT is subject to load testing while in step ② the response times are analyzed to identify possible performance gains.

5.1 | Load testing

In step ①, we inject the defined workload (the same used to extract the application graph model in Section 4.4), named **WL1**, to the SUT, the latter deployed as suggested by the deployment proposals obtained with the clustering techniques presented in Section 4.5.

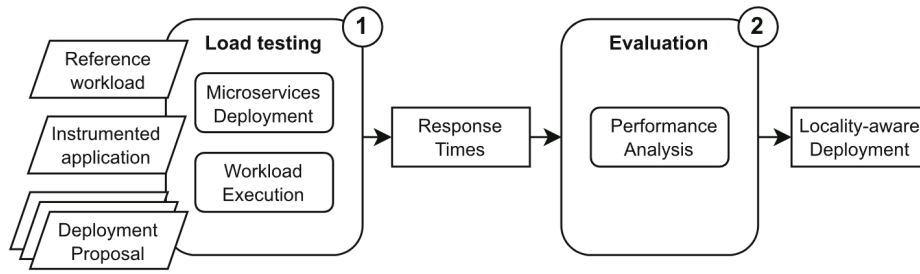


FIGURE 6 Deployment proposals validation.

To answer question **Q1**, we consider three additional random placements, labeled **R1**, **R2**, and **R3**. Random placement is obtained simply by selecting the machine on which to deploy each microservice uniformly at random, without a specific grouping strategy.

To answer question **Q2**, we consider two distinct scenarios: in the first one, we inject **WL1** to the SUT, with the latter being deployed as suggested by the deployment proposals, generated via clustering, considering both the original weighted WL-A model and its unweighted version, which is obtained by setting all the edge weights to 1; in the second, we take into account a significant modification of the previous workload, which leads to a new one, referred as **WL2**. We inject it into the SUT, extract the new WL-A model, apply clustering, and generate new deployment proposals that we then evaluate with respect to those obtained previously by injecting again **WL1** into the SUT.

The requests composing a workload are submitted sequentially, without concurrency to avoid uncontrollable interleaving also influenced by the probabilistic nature of the workload. Two experiments would indeed differ in the order in which the requests characterizing the selected behaviors are submitted. However, with one user we can record a specific sequence of requests and replay it with exactly the same interleaving in all the different tests. In this way, each request sees exactly the same state of the SUT in every test. This approach enables a fair comparison between the response times measured during the diverse experiments and it allows us to neglect the influence of the execution environments (available computational resources) and to focus on the impact of locality.

From the point of view of the load injector, the defined workloads (**WL1** and **WL2**) are characterized by a fixed total number of requests (10,000 in our case) such as to guarantee (i) a sufficient data sample size and (ii) a good coverage in terms of behaviors, with an acceptable test duration. Finally, every test is performed by considering both network settings introduced in Section 4.3 (**F**, **S**). Furthermore, for every setup, we perform multiple runs. We then select the run with the least fluctuation in response times. Specifically, we select the one that produces the sample with the greatest number of operations for which we have the smallest Interquartile Range (IQR). Raw data obtained during the experiments are available in a GitHub repository.^{¶¶}

5.2 | Test results evaluation

Step ② involves comparing the response times obtained in every test setup, with the aim of evaluating the impact of the locality for the diverse deployment proposals, with respect to the different dimensions targeted by the two questions introduced at the beginning of the section, consequently validating the approach and defining the module for automatic replacement of microservices. First, the exposed endpoints (see Table 1) are characterized in terms of the degree of sensitivity to the locality by calculating the amount of the response time spent in communicating with other endpoints.

Specifically, indicating with rt_j^i the j -th response time obtained by invoking the generic exposed endpoint e^i , the following score is defined:

$$r^i = \frac{\sum_{j=0}^{J^i} \left[\left(\frac{h_j^{i, 2 \cdot l^N}}{rt_j^i} \right) \cdot \alpha_{h_j^i} \right]}{\sum_{j=0}^{J^i} \alpha_{h_j^i}} = 2 \cdot l^N \cdot \frac{\sum_{j=0}^{J^i} \left[\left(\frac{h_j^i}{rt_j^i} \right) \cdot \alpha_{h_j^i} \right]}{\sum_{j=0}^{J^i} \alpha_{h_j^i}}, \quad (8)$$

^{¶¶}<https://github.com/DistLab-Sannio/DTLA-Data>.

where J^i is the number of times e^i is invoked, h_j^i is the number of calls that occur between pairs of endpoints belonging to different microservices and that are induced by the invocation of e^i , l^N is the latency characterizing the network N used by the execution environment hosting the application, and $\alpha_{h_j^i}$ is the number of times that calling e^i produces a chain of invocations with h_j^i hops; the multiplication by 2 takes into account both the request and response paths. This score is essentially the average of the communication latency percentages (the quantity inside the round brackets in the version of Equation (8) on the left), weighted by the frequency at which the specific numbers of hops occur (the $\alpha_{h_j^i}$ coefficient in Equation (8)). In the above definition, the latency is assumed to be the same for every hop. This choice is motivated by the impossibility of estimating the actual latency through the analysis of the temporal information present in the spans.^{##} Furthermore, we neglect the time that the load injector requires to send requests and get responses. For a better understanding, consider the example in which endpoint ab_1 is called three times. Assuming that (i) the invocations produce response times of 15, 25, and 17 ms, and chains with 3, 5, and 3 hops, respectively, and (ii) the latency is equal to 1ms, we have $r^{ab_1} = [(3 \times 2 \times 1/15) \times 2 + (5 \times 2 \times 1/25) \times 1 + (3 \times 2 \times 1/17) \times 2]/(2 + 1 + 2) = 0.382$. This means that approximately 38% of response time is spent communicating.

Second, the invocation frequency of the endpoints is considered and used to refine r as follows:

$$t^i = \frac{r^i \cdot J^i}{\sum_i J^i} \quad (9)$$

The idea is to give more importance to those endpoints that are invoked a lot in the context of a specific workload. Subsequently, for each pair of deployment proposals (D_{C_j}, D_{C_k}), we define a further score ρ_{C_j, C_k} (Equation (10)) based on the effect size η and p -value π obtained by the application of the Wilcoxon signed-rank test to the response times samples corresponding to the two proposals (for each operation). In light of the considerations reported in Section 5.1, we can indeed consider the sample containing the response times for a given endpoint in a specific deployment setting D_{C_j} , paired with that containing the response times for the same endpoint in another specific deployment setting D_{C_k} .

$$\rho_{C_j, C_k} = \frac{\sum_i \beta_{j,k}^i \times \eta_{j,k}^i \times t^i}{\sum_i 100 \times t^i} \quad (10)$$

In particular, t^i is the score defined in Equation (9) for the endpoint e^i , $\eta_{j,k}^i$ is the effect size deriving from the application of the Wilcoxon signed-rank test to the samples containing the response times for the endpoint e^i obtained by deploying the SUT according to D_{C_j} and D_{C_k} , and $\beta_{j,k}^i$ is a factor introduced to properly take $\eta_{j,k}^i$ into account. The latter is defined as follows:

$$\beta_{j,k}^i = \begin{cases} 1 & \text{if } (\eta_{j,k}^i > 0 \wedge \pi_{j,k}^i \leq 0.05) \vee (\eta_{j,k}^i < 0 \wedge \pi_{j,k}^i \geq 0.95) \\ 0 & \text{otherwise} \end{cases} \quad (11)$$

A positive effect size associated with a p -value below the significance level (0.05 in our case) suggests that D_{C_j} outperforms D_{C_k} ; conversely, a negative effect size associated with a p -value above 1 minus the significance level (0.95 in our case) suggests that D_{C_k} outperforms D_{C_j} . This interpretation derives from the way the statistical hypothesis test works with respect to the role of the operands, that is, which sample is the minuend and which sample is the subtrahend, and the alternative hypothesis, that is, test whether the distribution of the differences is stochastically less or greater than a distribution symmetric about zero. In our case, given the pair of samples s_j^i and s_k^i , both relating to endpoint e^i and corresponding to the deployments D_{C_j} and D_{C_k} , respectively, we consider the difference $s_k^i - s_j^i$ and the “greater” alternative hypothesis. Consequently, under these settings, having a negative effect size with a high p -value is equivalent to having a positive effect size with a low p -value, but with the operands of the differences reversed ($s_j^i - s_k^i$).

The effect size ranges from -100 to 100 . Consequently, using 100 in the denominator of Equation (10) leads to a score that ranges from -1 (worst case) to 1 (ideal case). From an operational perspective, we calculate r by deploying the whole SUT in Worker-5 and by configuring the load injector to consecutively perform the sequences of $WL1$ for

^{##}Spans only contain the timestamp of when they start in the callee, but not when they are suspended in the caller.

TABLE 3 Effect sizes (η) and p -values (π) of the Wilcoxon signed-rank tests between cluster-based and random-based samples for the top-10 operations (the first 10 operations with the highest t score).

Deployment proposal																					
Op.	t	SP-R1		SP-R2		SP-R3		ARC-R1		ARC-R2		ARC-R3		LA-R1		LA-R2		LA-R3			
		$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$	$\eta^i_{j,k}$	$\pi^i_{j,k}$				
(a) S network																					
to ₁	0.785	100.00	0.00	100.00	0.00	100.00	0.00	99.99	0.00	100.00	0.00	100.00	0.00	100.00	0.00	100.00	0.00	100.00	0.00		
tp ₁	0.308	100.00	0.00	100.00	0.00	100.00	0.00	99.98	0.00	100.00	0.00	100.00	0.00	100.00	0.00	100.00	0.00	100.00	0.00		
as ₁	0.245	99.24	0.00	99.97	0.00	99.78	0.00	98.48	0.00	98.98	0.00	98.97	0.00	31.57	0.00	1.17	0.36	12.32	0.00		
th ₁	0.208	96.85	0.00	99.99	0.00	100.00	0.00	-96.51	1.00	99.79	0.00	100.00	0.00	99.58	0.00	100.00	0.00	100.00	0.00		
co ₁	0.184	98.92	0.00	98.90	0.00	98.94	0.00	97.47	0.00	97.96	0.00	97.99	0.00	98.76	0.00	99.01	0.00	98.56	0.00		
fo ₁	0.159	19.15	0.00	95.04	0.00	37.38	0.00	93.58	0.00	98.44	0.00	95.11	0.00	30.38	0.00	96.73	0.00	48.81	0.00		
po ₁	0.155	98.90	0.00	98.13	0.00	-68.27	1.00	98.59	0.00	98.67	0.00	9.83	0.02	71.03	0.00	75.11	0.00	-93.80	1.00		
ip ₁	0.105	96.99	0.00	97.18	0.00	97.97	0.00	96.21	0.00	96.26	0.00	97.54	0.00	8.61	0.02	9.58	0.01	29.01	0.00		
oo ₁	0.046	-97.06	1.00	-97.51	1.00	0.24	0.43	-97.12	1.00	-97.85	1.00	3.62	0.18	44.54	0.00	30.21	0.00	97.87	0.00		
ph ₁	0.038	96.02	0.00	95.82	0.00	82.72	0.00	99.62	0.00	97.91	0.00	97.00	0.00	82.19	0.00	84.87	0.00	37.03	0.00		
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...		
ρ_{C_j,C_k}		0.872		0.925		0.816		0.764		0.940		0.907		0.765		0.779		0.688			
(b) F network																					
to ₁	0.078	99.99	0.00	99.95	0.00	99.92	0.00	99.84	0.00	98.95	0.00	98.59	0.00	99.86	0.00	99.62	0.00	99.59	0.00		
tp ₁	0.031	99.99	0.00	100.00	0.00	100.00	0.00	95.44	0.00	99.25	0.00	99.88	0.00	99.11	0.00	99.82	0.00	99.98	0.00		
as ₁	0.024	77.50	0.00	64.34	0.00	69.92	0.00	71.08	0.00	56.06	0.00	57.52	0.00	-5.83	0.91	-28.08	1.00	-29.79	1.00		
th ₁	0.021	52.80	0.00	99.52	0.00	99.56	0.00	-77.98	1.00	97.55	0.00	95.84	0.00	43.42	0.00	98.91	0.00	99.56	0.00		
co ₁	0.018	66.36	0.00	57.77	0.00	53.19	0.00	53.37	0.00	45.64	0.00	35.87	0.00	87.70	0.00	82.89	0.00	78.89	0.00		
fo ₁	0.016	-4.87	0.87	47.66	0.00	-32.05	1.00	14.77	0.00	55.04	0.00	-9.70	0.99	33.23	0.00	72.27	0.00	2.92	0.24		
po ₁	0.015	60.88	0.00	46.69	0.00	-69.39	1.00	80.20	0.00	72.79	0.00	-31.94	1.00	52.14	0.00	39.52	0.00	-74.38	1.00		
ip ₁	0.010	41.54	0.00	41.83	0.00	36.51	0.00	62.50	0.00	63.84	0.00	61.53	0.00	-44.66	1.00	-36.73	1.00	-46.51	1.00		
oo ₁	0.005	-61.27	1.00	-62.12	1.00	-17.25	1.00	-59.64	1.00	-56.11	1.00	-8.35	0.98	10.02	0.01	7.47	0.03	61.03	0.00		
ph ₁	0.004	44.31	0.00	42.54	0.00	-19.59	0.98	74.37	0.00	70.27	0.00	20.17	0.01	49.72	0.00	50.74	0.00	-15.61	0.96		
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...		
ρ_{C_j,C_k}		0.705		0.752		0.621		0.616		0.779		0.663		0.615		0.656		0.530			

Note: The last row reports the ρ score.

a certain number of times (50 in our case), without considering their probability. In this way, we can characterize the endpoints in terms of communication latency. The response times are retrieved directly from the spans; specifically, from the *duration* field of those that are associated with invocations of the exposed endpoints on the front-end proxy. The t score is instead calculated by considering the actual invocation frequencies of the endpoints, which depend on the probabilities associated with the different behaviors. Finally, the ρ score is calculated starting from the response times collected by the load injector.

As for question **Q1**, in Table 3 we report the effect sizes and p -values of the Wilcoxon signed-rank tests between cluster-based and random-based samples for the top-10 endpoints, that is, the first 10 endpoints with the highest t , along with ρ values for the considered pairs. The latter are visually represented in Figure 7.

With both **S** and **F** networks, cluster-based solutions outperform random ones. The best deployment proposals are **LA** with **S** network and **SP** with **F** network. As shown, the impact of locality significantly increases as the performance of the network infrastructure decreases. In fact, with the **S** network, ρ is always higher than with **F** network. To give an idea of the performance gain achieved, we observed the following response times (95th percentile) for the most important endpoint (to_1): 1087.200ms for **ARC**, 777.369ms for **SP**, 563.037ms for **LA**, 1900.590ms for **R1**, 1851.859ms for **R2** and 1431.974ms for **R3**, with the **S** network, and 434.538ms for **ARC**, 396.661ms for **SP**, 391.254ms for **LA**, 541.805ms for **R1**, 551.226ms for **R2**, and 497.759ms for **R3**, with the **F** network. Considering the best alternatives among the two groups (**LA** and **R3**) we obtained a reduction of 61% for **S** network and 21% for **F** network.

To answer **Q2**, in the first scenario, we calculated the following scores: $\rho_{SP,SP_U} = 0.495$ and $\rho_{LA,LA_U} = 0.560$ with **S** network, and $\rho_{SP,SP_U} = 0.470$ and $\rho_{LA,LA_U} = 0.484$ with **F** network. The suffix “ $_U$ ” refers to the deployment proposals generated by applying the clustering techniques to the unweighted version of the WL-A model. The obtained values indicate that being aware of interaction intensities actually leads to performance improvements. Note that we do not show the score for the **ARC** technique because the corresponding configuration for the unweighted model was not feasible in terms of resource consumption. For the second scenario, we calculated the following scores: $\rho_{SP_WL1,SP_WL2} = 0.621$ and $\rho_{LA_WL1,LA_WL2} = 0.697$ with **S** network, and $\rho_{SP_WL1,SP_WL2} = 0.387$ and $\rho_{LA_WL1,LA_WL2} = 0.621$ with **F** network. Suffixes “ $_WL1$ ” and “ $_WL2$ ” refer to the deployment proposals generated via clustering techniques applied to the WL-A models extracted by injecting **WL1** and **WL2** to the SUT, respectively, but evaluated by injecting **WL1** in both cases.

The obtained values confirm that adapting the deployment to the workload changes reduces response times and increases performance. Even in this case, we do not show the score for the **ARC** technique because the corresponding configuration for the model extracted by injecting **WL2** was not feasible from the point of view of resource consumption.

It is worth noting that the defined scores provide an overall evaluation of the deployment proposals: having one that behaves better than another does not mean that there is an improvement for every endpoint. In fact,

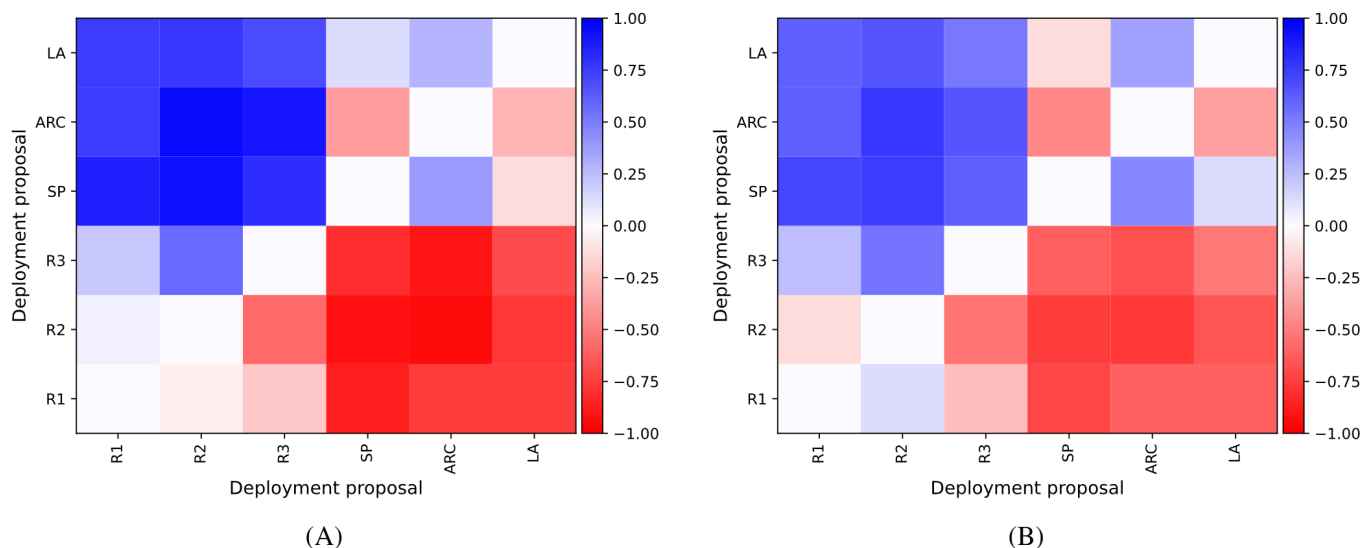


FIGURE 7 Visualization of ρ scores. (A) **S** network. (B) **F** network.

breaking the whole application into groups inevitably leads some local, that is, intra-group, interactions to become non-local, that is, inter-group. Locality-oriented clustering techniques try to reduce their number, which however remains non-zero. As a consequence, there will be cross-cluster call paths that will be penalized by the identified deployment.

6 | DISCUSSION

Defining an optimal deployment of microservices-based applications is a challenging problem, due to the number of variables that have to be taken into account and that come from (i) the hosting (physical and/or virtualized) infrastructure, and (ii) the peculiar characteristics of the applications themselves, along with the specific workload to which they are subjected.

Test results demonstrate that locality-awareness helps improve performance, but its effects largely depend on the workload, the structure of the application, and the hardware configuration. The impact of locality, in fact, grows when the application granularity decreases and this could depend on both the application structure and the load injected w.r.t. the adopted computing resources. Therefore, a static analysis is insufficient to understand the impact of communication and the benefits of locality-awareness; on the other hand, a dynamic analysis, conducted on the basis of usage metrics and tracing data gathered at run-time, could provide useful insights about possible re-configurations to improve performance (e.g., when the computation load increases and it is not possible to scale further the system by adding other computing resources or when latency becomes dominant).

For example, in addition to the experiments presented in Section 5, we also verified whether the results obtained with one user also occur under load. In particular, we repeated the experiments relating to the analysis of the locality-awareness with a number of users equal to 20 and 40, still keeping in mind what was said in Section 5.1. Only for this specific case, we have redefined ρ as follows:

$$\rho'_{C_j, C_k} = \frac{\sum_i \gamma_{j,k}^i * t^i}{\sum_i t^i} \quad (12)$$

In particular, t^i is again the score defined in Equation (9) for the endpoint e^i , and γ_i is a factor introduced to properly take t^i into account. Specifically, it is defined as follows:

$$\gamma_{j,k}^i = \begin{cases} 1 & \text{if } p\text{-th percentile of response time of } e^i \text{ for } D_{C_j} \text{ is lower than that for } D_{C_k} \\ -1 & \text{otherwise} \end{cases} \quad (13)$$

We consider $p = 95$. With this definition, we have again a score ranging from -1 (worst case) to 1 (best case). In Figure 8 we report the visual representation of the redefined ρ score in the different settings.

With the **S** network, the results are the same as those observed during the validation phase, either with 20 or 40 users. Specifically, the cluster-based deployments behave better than the random ones, with **LA** being the best solution. However, with **F** network, when the load increases from 20 to 40 users, the performance of the first group degrades. In particular, **LA** becomes the worst choice. This is due to the increased concurrency that led to a significant unbalancing between computational (CPU and RAM) and communication (network) resources. In fact, as shown in Figure 9, by increasing the amount of available CPUs and RAM, **LA** regains the position of the best solution (see Figure 9A). Note that also the random strategies could benefit from the resizing operation. For example, **R3** improves over the other techniques, except for **LA**. However, in some cases, CPU and RAM do not represent a problem. For instance, for **SP** the bottleneck is the network bandwidth. Increasing the latter allows it to win against **R2** and **R3**, but not **R1**, which benefits from the scaling operation in the same way (see Figure 9B). The reason for this behavior lies in the increased amount of inter-cluster calls that the **SP** and **R1** deployments lead to.

The considerations above bring to the need for continuous monitoring of the workload (by logging requests) in order to have an up-to-date reference workload to inject into a twin of the system to extract the WL-A graph model for exploring the existence of better placements based on locality, according to which to re-configure the system in production. Using the

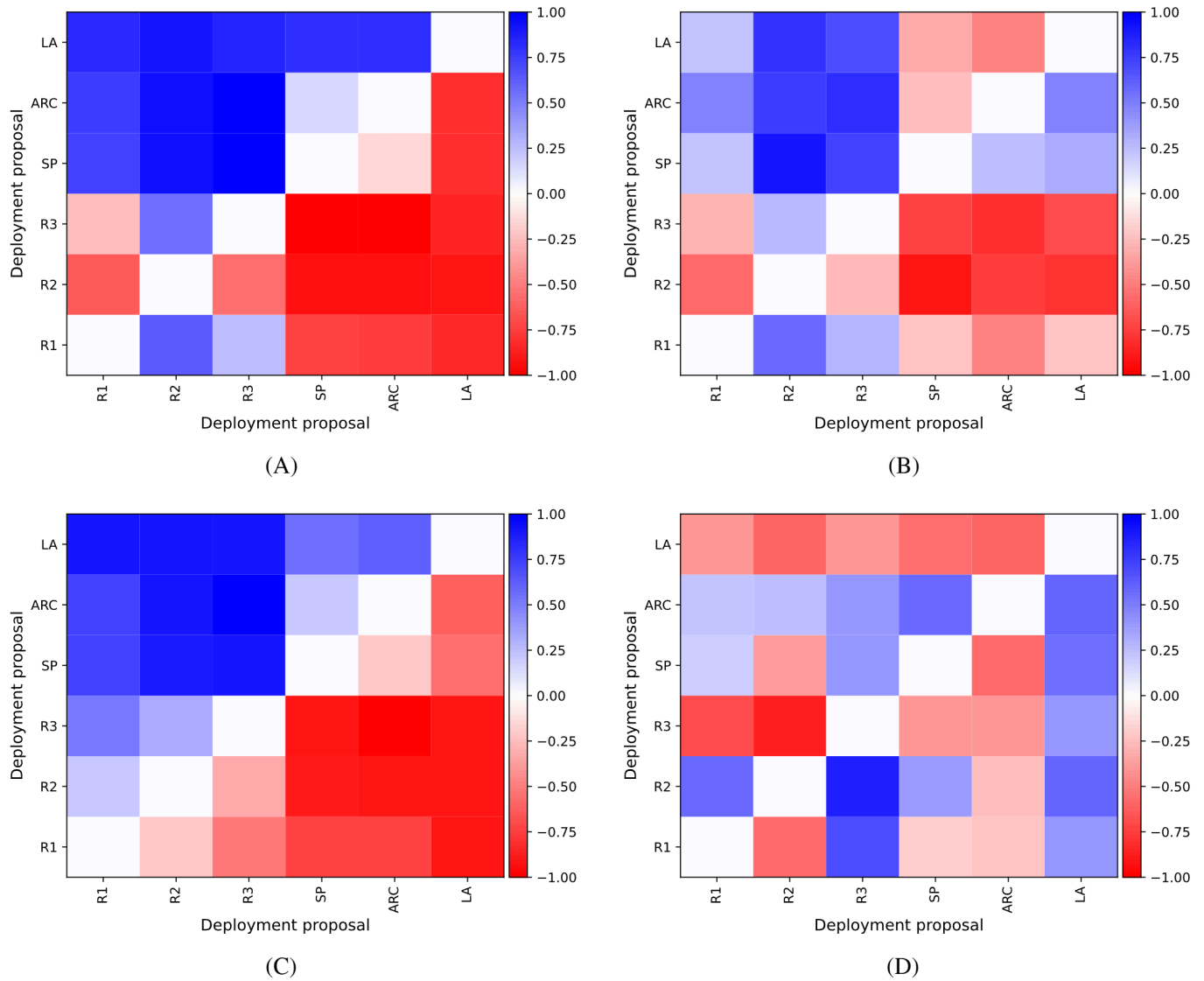


FIGURE 8 Visualization of ρ' scores with 20 and 40 users, **S** and **F** networks. (A) **S** network and 20 concurrent users. (B) **F** network and 20 concurrent users. (C) **S** network and 40 concurrent users. (D) **F** network and 40 concurrent users.

twin avoids perturbing the system with additional delays in response times. Note that the twin does not need to be online all the time; it can be activated only when the analysis has to be carried out (e.g., after a new deployment of a DevOps pipeline or when response times increase even in presence of replication). The proposed configuration is equivalent to a typical one characterized by the presence of both test and production environments.

The ρ score introduced in Equation (10) has been defined to analyze in detail the aspects relating to locality-awareness and workload-awareness, with the aim of validating the proposed framework. Despite this, it cannot always be used to select the best deployment proposal. For example, when we addressed question **Q1** during the validation, we said that **LA** was the best solution with **S** network. This statement was based on the fact that **LA** won against all the other proposals. Basically, we considered all pairwise comparisons. However, there may be cases in which the best solution cannot be identified. This becomes critical, especially in a DevOps context, where the selection of the deployment proposal should be made automatically. For this reason, for each proposal, we introduce a new score, called σ , defined in Equation (14).

$$\sigma_{C_k} = \frac{\sum_i \sum_{j=0}^{J^i} \delta_{k,j}^i \times t^i}{\sum_i t^i \times J^i} \quad (14)$$

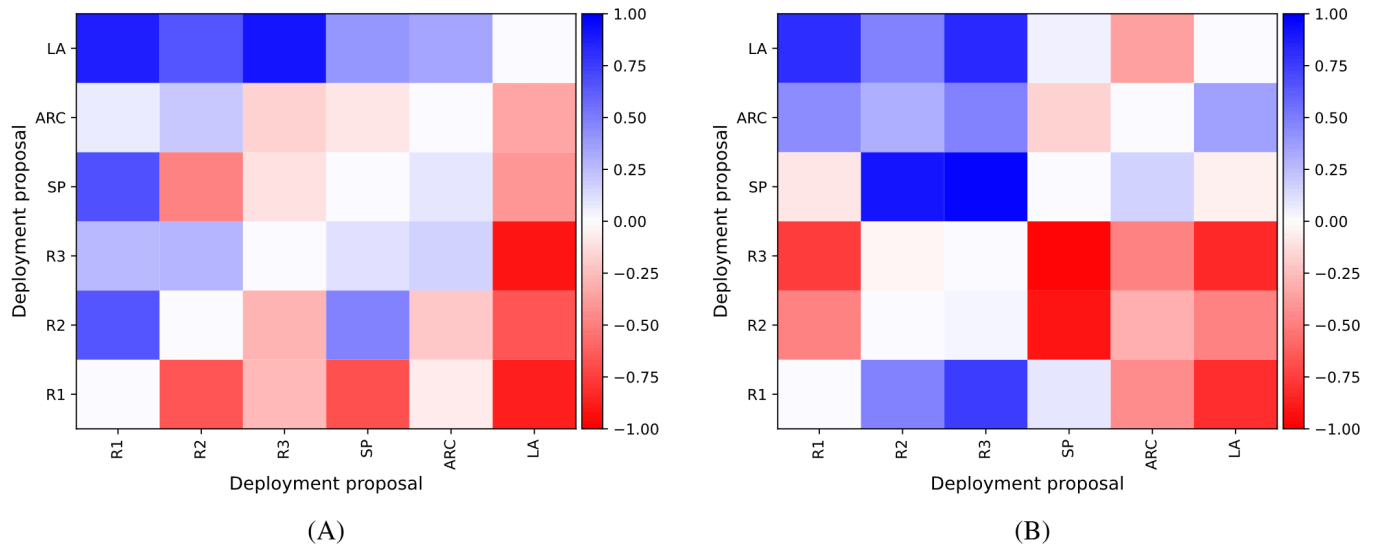


FIGURE 9 Visualization of p' scores with 40 users, **F** network and increased resources. (A) Increased CPU and RAM. (B) Increased CPU, RAM and network bandwidth.

In particular, t^i is the score defined in Equation (9) for the endpoint e^i , J^i is the number of times e^i is invoked in the context of a specific workload, and $\delta_{k,j}^i$ is a factor defined in Equation (15) to properly take into account t^i .

$$\delta_{k,j}^i = \begin{cases} 1 & \text{if } RT_k^i[j] \leq RT_l^i[j], \quad \forall l \in \{1, \dots, K\} - k \\ 0 & \text{otherwise} \end{cases} \quad (15)$$

Assuming K deployment proposals, referred with integers belonging to the interval $[1, K]$, and indicating with $RT_k^i = \{rt_j^i\}_{j=0}^{J^i}$ the sample containing the response times for the endpoint e^i obtained with the deployment k , we have that $RT_k^i[j]$ is the j -th response time obtained by invoking e^i with the deployment k . Consequently, $\delta_{k,j}^i$ is equal to 1 when the j -th response time relating to e^i is minimal with deployment k . In the worst case, that is, when all response times of an endpoint obtained with a specific deployment proposal are never minimal, we obtain 0; in the best case, that is, when all response times of an endpoint obtained with a specific deployment proposal are always minimal, we obtain $J^i \times t^i$. Consequently, σ ranges from 0 to 1. The best solution is therefore the one with the highest σ value. Calculating it for the case analyzed when addressing question **Q1**, we obtain the following values, which lead to the same considerations made during the validation phase: $\sigma_{SP} = 0.201$, $\sigma_{ARC} = 0.177$, $\sigma_{LA} = \mathbf{0.582}$, $\sigma_{R1} = 0.009$, $\sigma_{R2} = 0.006$, $\sigma_{R3} = 0.024$ with **S** network, and $\sigma_{SP} = \mathbf{0.396}$, $\sigma_{ARC} = 0.137$, $\sigma_{LA} = 0.357$, $\sigma_{R1} = 0.027$, $\sigma_{R2} = 0.024$, $\sigma_{R3} = 0.058$ with **F** network.

Some final considerations concern the usability of the proposed approach. During our experiments, we kept track of the time necessary to (i) extract the WL-A model, (ii) generate the deployment proposals via clustering, and (iii) analyze the corresponding response times. In the first case, with a custom parallel script, we observed a running time of about 20 s to extract the model by processing the 10,000 traces related to the same number of requests, which consist of approximately 2.5 million spans (just over 500 MB in MongoDB). In the second case, we were able to generate all deployment proposals in less than half a minute. The same time was finally required in the third case to evaluate all response times through the computation of the σ score with a custom sequential script (which can be easily made parallel). Of course, for each proposal, we need to properly re-deploy the SUT and inject the workload. As for TrainTicket, it took about 3 min to be deployed and ready, while the load testing lasted approximately 1 h on average for the **S** network (with a maximum of 1.35 h, corresponding to **R2**) and 23.4 min on average for the **F** network (with a maximum of 25.6 min, corresponding to **R2**). The values obtained depend on (i) the size of the application, that is, the number of microservices, (ii) the amount of interactions, that is, the number of spans, (iii) the number of deployment strategies, (iv) the number of endpoints, and (v) the number of requests injected. Having tested our approach on a realistic medium-sized system, we believe that the observed overhead is acceptable. As a possible use case, Cloud providers may exploit a framework like the one proposed in this work *as a service*, for example, to negotiate different QoS levels.

7 | CONCLUSION

In this paper, we proposed a framework for microservices placement that exploits a big data pipeline to collect and analyze telemetry data generated by injecting a specific workload. This can be derived from execution logs or by considering a typical usage of the system, depending on whether the system itself is in production or not. Telemetry data is then processed to construct a graph-based runtime model for the specific workload used as input for diverse locality-aware clustering techniques, each one responsible for identifying k groups of microservices to be deployed on as many execution environments.

The proposed framework has been applied to TrainTicket, and the different deployment proposals generated for this particular case study have been evaluated. We assessed the performance gain through the definition of a set of metrics that allowed us to take into consideration the impact of the locality on the operations exposed.

The experimental results showed that adopting locality-aware strategies can effectively decrease response times, especially when the amount of computational resources is sufficient to compete with the performance of the underlying network infrastructure.

As for future works, we plan to extend the framework by (i) increasing the granularity of the analysis, that is, by considering the additional layers of the graph model related to the operation, and (ii) investigating its employment as a tool to support application refactoring and scaling mechanisms. In addition, we target the application of our proposal to different classes of microservices-based applications. Finally, we will work to integrate the proposed framework into a DevOps toolchain.

AUTHOR CONTRIBUTIONS

Carmine Colarusso: Conceptualization, software, investigation, validation, writing – original draft. **Assunta De Caro:** Conceptualization, data curation, validation, writing – original draft. **Ida Falco:** Conceptualization, software, validation, writing – original draft. **Lorenzo Goglia:** Conceptualization, data curation, validation, formal-analysis, writing – original draft. **Eugenio Zimeo:** Conceptualization (lead), validation, formal-analysis, writing – review and editing, supervision.

ACKNOWLEDGMENTS

This work has been partially supported by the SISMA (SolutIonS for engineering Microservices Applications) project funded by Ministero dell'Università e della Ricerca [grant number 201752ENYB_002].

CONFLICT OF INTEREST STATEMENT

The authors declare no potential conflict of interests.

DATA AVAILABILITY STATEMENT

All raw data collected during our evaluation are publicly available at: <https://github.com/DistLab-Sannio/DTLA-Data>.

ORCID

Eugenio Zimeo  <https://orcid.org/0000-0003-4683-5487>

REFERENCES

1. MartinFowler. Microservices. <https://martinfowler.com/articles/microservices.html> 2023.
2. De Iasio A, Zimeo E. A framework for microservices synchronization. *Softw Pract Exp*. 2021;51(1):25–45. doi:10.1002/spe.2877
3. Camilli M, Colarusso C, Russo B, Zimeo E. Actor-driven decomposition of microservices through multi-level scalability assessment. *ACM Trans Softw Eng Methodol*. 2023;32(5):1–46. doi:10.1145/3583563
4. <https://microservices.io/patterns/observability/distributed-tracing.html>. 2023.
5. Joseph CT, Chandrasekaran K. IntMA: dynamic interaction-aware resource allocation for containerized microservices in cloud environments. *J Syst Archit*. 2020;111:101785. doi:10.1016/j.sysarc.2020.101785
6. Sampaio Junior A, Rubin J, Beschastnikh I, Rosa N. Improving microservice-based applications with runtime placement adaptation. *J Internet Serv Appl*. 2019;10(4):1–30. doi:10.1186/s13174-019-0104-0
7. Sakai M, Takahashi K. Constructing a service process model based on distributed tracing for conformance checking of microservices. Paper presented at: NOMS 2022-2022 IEEE/IFIP Network Operations and Management Symposium. IEEE Press. 2022 1–6.
8. Cassé C, Berthou P, Owezarski P, Josset S. Using distributed tracing to identify inefficient resources composition in cloud applications. Paper presented at: 2021 IEEE 10th International Conference on Cloud Networking (CloudNet). 2021 40–47.

9. Casse C, Berthou P, Owezarski P, Josset S. A tracing based model to identify bottlenecks in physically distributed applications. Paper presented at: 2022 International Conference on Information Networking (ICOIN). IEEE Computer Society; Los Alamitos, CA, USA. 2022 226-231.
10. Chen H, Wei K, Li A, Wang T, Zhang W. Trace-based intelligent fault diagnosis for microservices with deep learning. Paper presented at: 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC). 2021 884-893.
11. Bento A, Correia J, Filipe R, Araujo F, Cardoso J. Automated analysis of distributed tracing: challenges and research directions. *J Grid Comput*. 2021;19(1):1-15. doi:[10.1007/s10723-021-09551-5](https://doi.org/10.1007/s10723-021-09551-5)
12. Maruf AA, Bakhtin A, Cerny T, Taibi D. Using microservice telemetry data for system dynamic analysis. Paper presented at: 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE). IEEE Computer Society; Los Alamitos, CA, USA. 2022 29-38.
13. Bogatinovski J, Nedelkoski S, Cardoso J, Kao O. Self-supervised anomaly detection from distributed traces. Paper presented at: 2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC). 2020 342-347.
14. Han J, Hong Y, Kim J. Refining microservices placement employing workload profiling over multiple Kubernetes clusters. *IEEE Access*. 2020;8:192543-192556. doi:[10.1109/ACCESS.2020.3033019](https://doi.org/10.1109/ACCESS.2020.3033019)
15. Zhao D, Mohamed M, Ludwig H. Locality-aware scheduling for containers in cloud computing. *IEEE Trans Cloud Comput*. 2020;8(2):635-646. doi:[10.1109/TCC.2018.2794344](https://doi.org/10.1109/TCC.2018.2794344)
16. Saboor A, Mahmood AK, Omar AH, Hassan MF, Shah SNM, Ahmadian A. Enabling rank-based distribution of microservices among containers for green cloud computing environment. *Peer Peer Netw Appl*. 2022;15(1):77-91. doi:[10.1007/s12083-021-01218-y](https://doi.org/10.1007/s12083-021-01218-y)
17. Hu Y, Laat C, Zhao Z. Optimizing service placement for microservice architecture in clouds. *Appl Sci*. 2019;9(21):1-18. doi:[10.3390/app9214663](https://doi.org/10.3390/app9214663)
18. Ding Z, Wang S, Jiang C. Kubernetes-oriented microservice placement with dynamic resource allocation. *IEEE Trans Cloud Comput*. 2023;11(2):1777-1793. doi:[10.1109/TCC.2022.3161900](https://doi.org/10.1109/TCC.2022.3161900)
19. Liang H, Chou J. HPA: hierarchical placement algorithm for multi-cloud microservices applications. Paper presented at: 2022 IEEE International Conference on Cloud Computing Technology and Science (CloudCom). 2022 17-24.
20. Sheoran A, Fahmy S, Sharma P, Modi N. Invenio: communication affinity computation for low-latency microservices. Paper presented at: ANCS'21. Proceedings of the Symposium on Architectures for Networking and Communications Systems. Association for Computing Machinery; New York, NY, USA. 2022 88-101.
21. Zaheer S, Malik A, Rahman AU, Khan S. Locality-aware process placement for parallel and distributed simulation in cloud data centers. *J Supercomput*. 2019;75(11):7723-7745. doi:[10.1007/s11227-019-02973-9](https://doi.org/10.1007/s11227-019-02973-9)
22. Pothan A, Simon HD, Liou KP. Partitioning sparse matrices with eigenvectors of graphs. *SIAM J Matrix Anal Appl*. 1990;11(3):430-452. doi:[10.1137/0611030](https://doi.org/10.1137/0611030)
23. Damle A, Minden V, Ying L. Simple, direct and efficient multi-way spectral clustering. *Inf Inference: J IMA*. 2018;8(1):181-203. doi:[10.1093/imaiai/iay008](https://doi.org/10.1093/imaiai/iay008)
24. Mones E, Vicsek L, Vicsek T. Hierarchy measure for complex networks. *PloS One*. 2012;7:e33799. doi:[10.1371/journal.pone.0033799](https://doi.org/10.1371/journal.pone.0033799)

How to cite this article: Colarusso C, De Caro A, Falco I, Goglia L, Zimeo E. A distributed tracing pipeline for improving locality awareness of microservices applications. *Softw: Pract Exper*. 2024;54(6):1118-1140. doi: [10.1002/spe.3317](https://doi.org/10.1002/spe.3317)